

```

from datetime import datetime, timedelta
import logging
import sqlite3
import requests
import telebot
from telebot import types

# Логирование
logging.basicConfig(level=logging.INFO)

# Ваши данные
BOT_TOKEN = "8956045323:AAFTOBqakxpHQm8tkiLwcYmyUCeoPyh425Q"
PIARFLOW_API_KEY = "SHvwjKtASrhS2a_7y9PK6ykSnh_kFBle"
PIARFLOW_BASE_URL = "https://piarflow.com/v1"

# Твой ID администратора
ADMIN_ID = 5874003289

bot = telebot.TeleBot(BOT_TOKEN)

# Заголовки для авторизации в PiarFlow
headers = {
    "Authorization": f"Bearer {PIARFLOW_API_KEY}",
    "Content-Type": "application/json",
}

# Словарь для отслеживания состояний пользователей (ввод суммы, ввод ID для админки)
user_states = {}
admin_temp_data = {} # Для временного хранения данных админа (кого банят/кому меняют баланс)

# --- НАСТРОЙКА БАЗЫ ДАННЫХ ---
conn = sqlite3.connect("bot_database.db", check_same_thread=False)
cursor = conn.cursor()

# Создаем таблицу пользователей (добавлено поле is_banned)
cursor.execute("""
CREATE TABLE IF NOT EXISTS users (
    user_id INTEGER PRIMARY KEY,
    balance INTEGER DEFAULT 0,
    last_bonus TEXT,
    referrer_id INTEGER,
    is_banned INTEGER DEFAULT 0
)
""")
conn.commit()

```

```

def get_user(user_id):
    cursor.execute(
        "SELECT balance, last_bonus, referrer_id, is_banned FROM users WHERE"
        " user_id = ?",
        (user_id,),
    )
    user = cursor.fetchone()
    if not user:
        cursor.execute(
            "INSERT INTO users (user_id, balance, last_bonus, referrer_id,"
            " is_banned) VALUES (?, 0, NULL, NULL, 0)",
            (user_id,),
        )
        conn.commit()
        return 0, None, None, 0
    return user[0], user[1], user[2], user[3]

```

```

def set_referrer(user_id, referrer_id):
    cursor.execute(
        "SELECT referrer_id FROM users WHERE user_id = ?", (user_id,)
    )
    res = cursor.fetchone()
    if res and res[0] is None and user_id != referrer_id:
        cursor.execute(
            "UPDATE users SET referrer_id = ? WHERE user_id = ?",
            (referrer_id, user_id),
        )
        conn.commit()

```

```

def update_user_balance_and_bonus(user_id, added_balance, bonus_time_str):
    cursor.execute(
        "UPDATE users SET balance = balance + ?, last_bonus = ? WHERE user_id ="
        " ?",
        (added_balance, bonus_time_str, user_id),
    )
    conn.commit()

```

```

def add_balance(user_id, amount):
    cursor.execute(
        "UPDATE users SET balance = balance + ? WHERE user_id = ?",
        (amount, user_id),
    )
    conn.commit()
    cursor.execute("SELECT balance FROM users WHERE user_id = ?", (user_id,))

```

```
return cursor.fetchone()[0]
```

```
def set_balance(user_id, amount):  
    cursor.execute(  
        "UPDATE users SET balance = ? WHERE user_id = ?", (amount, user_id)  
    )  
    conn.commit()
```

```
def set_ban_status(user_id, status):  
    cursor.execute(  
        "UPDATE users SET is_banned = ? WHERE user_id = ?", (status, user_id)  
    )  
    conn.commit()
```

```
def get_referral_count(user_id):  
    cursor.execute(  
        "SELECT COUNT(*) FROM users WHERE referrer_id = ?", (user_id,) )  
    return cursor.fetchone()[0]
```

Запрос спонсоров у PiarFlow

```
def get_sponsors(user_id, chat_id, max_sponsors=3):  
    url = f"{PIARFLOW_BASE_URL}/sponsors"  
    payload = {  
        "user_id": user_id,  
        "chat_id": chat_id,  
        "max_sponsors": max_sponsors,  
    }  
    try:  
        response = requests.post(url, json=payload, headers=headers, timeout=10)  
        if response.status_code == 200:  
            return response.json().get("sponsors", [])  
    except Exception as e:  
        print(f"Ошибка при запросе спонсоров: {e}")  
    return []
```

Проверка подписок

```
def check_sponsors(user_id, links):  
    url = f"{PIARFLOW_BASE_URL}/sponsors/check"  
    payload = {"user_id": user_id, "links": links}  
    try:  
        response = requests.post(url, json=payload, headers=headers, timeout=10)  
        if response.status_code == 200:
```

```
    return response.json().get("sponsors", [])
except Exception as e:
    print(f"Ошибка при проверке подписок: {e}")
return []
```

Главное меню с кнопками

```
def main_menu():
    markup = types.ReplyKeyboardMarkup(resize_keyboard=True, row_width=2)
    btn_profile = types.KeyboardButton("👤 Профиль")
    btn_bonus = types.KeyboardButton("🎁 Бонус")
    btn_ref = types.KeyboardButton("👥 Пригласить друга")
    markup.add(btn_profile, btn_bonus, btn_ref)
    return markup
```

Миддлварь/проверка на бан при любом сообщении

```
@bot.message_handler(
    func=lambda message: get_user(message.from_user.id)[3] == 1,
    content_types=[
        "text",
        "audio",
        "document",
        "photo",
        "sticker",
        "video",
        "voice",
    ],
)
def blocked_user_handler(message):
    if message.from_user.id == ADMIN_ID:
        return # Админа не бачим даже если статус 1 случайно
    bot.send_message(
        message.chat.id, "❌ Вы заблокированы администратором этого бота."
    )
```

Команда /start

```
@bot.message_handler(commands=["start"])
def start_cmd(message):
    user_id = message.from_user.id
    chat_id = message.chat.id
    balance, _, _, is_banned = get_user(user_id)

    if is_banned and user_id != ADMIN_ID:
        bot.send_message(chat_id, "❌ Вы заблокированы администратором этого бота.")
    return
```

```
user_states.pop(user_id, None)
```

```
args = message.text.split()
```

```
if len(args) > 1:
```

```
    try:
```

```
        ref_id = int(args[1])
```

```
        set_referrer(user_id, ref_id)
```

```
    except ValueError:
```

```
        pass
```

```
sponsors = get_sponsors(user_id, chat_id)
```

```
if not sponsors:
```

```
    bot.send_message(
```

```
        chat_id,
```

```
        "🔒 Спасибо за подписку!\n\n"
```

```
        "♥ Напоминаю: тут платят по 7.000 💰 GRAM за реферала\n\n"
```

```
        "✅ Приятного использования!",
```

```
        reply_markup=main_menu(),
```

```
    )
```

```
    return
```

```
keyboard = types.InlineKeyboardMarkup(row_width=1)
```

```
for i, sponsor in enumerate(sponsors, 1):
```

```
    link = sponsor.get("link")
```

```
    btn_link = types.InlineKeyboardButton(
```

```
        text=f"👉 Подписаться на Канал #{i}", url=link
```

```
    )
```

```
    keyboard.add(btn_link)
```

```
btn_check = types.InlineKeyboardButton(
```

```
    text="✅ Я подписался (Проверить)", callback_data="check_sub"
```

```
)
```

```
keyboard.add(btn_check)
```

```
bot.send_message(
```

```
    chat_id,
```

```
    "Для доступа к боту необходимо подписаться на каналы спонсоров ниже:",
```

```
    reply_markup=keyboard,
```

```
)
```

```
# Обработка успешного прохождения всех подписок
```

```
def handle_successful_subscription(user_id, chat_id, message_id=None):
```

```
    balance, _, referrer_id, _ = get_user(user_id)
```

```
    if referrer_id:
```

```
        new_ref_balance = add_balance(referrer_id, 7000)
```

```

try:
    user_info = bot.get_chat(user_id)
    who = (
        f"@{user_info.username}"
        if user_info.username
        else str(user_id)
    )
except:
    who = str(user_id)

try:
    bot.send_message(
        referrer_id,
        f"{who} Подписался на всех спонсоров по вашей ссылке!\n"
        f"Вы получили 7.000 💰\n"
        f"Ваш текущий баланс {new_ref_balance} Gram",
    )
except Exception as e:
    print(f"Не удалось отправить бонус рефереру: {e}")

cursor.execute(
    "UPDATE users SET referrer_id = NULL WHERE user_id = ?", (user_id,)
)
conn.commit()

success_text = (
    "🔒 Спасибо за подписку!\n\n"
    "♥️ Напоминаю: тут платят по 7.000 💰 GRAM за реферала\n\n"
    "✅ Приятного использования!"
)

if message_id:
    bot.edit_message_text(
        success_text, chat_id=chat_id, message_id=message_id
    )
else:
    bot.send_message(chat_id, success_text)

bot.send_message(chat_id, "Главное меню:", reply_markup=main_menu())

# Проверка подписок через инлайн-кнопку
@bot.callback_query_handler(func=lambda call: call.data == "check_sub")
def callback_check(call):
    user_id = call.from_user.id
    chat_id = call.message.chat.id

    sponsors = get_sponsors(user_id, chat_id)

```

```

if not sponsors:
    bot.answer_callback_query(call.id, "Все задания выполнены!")
    handle_successful_subscription(
        user_id, chat_id, message_id=call.message.message_id
    )
    return

links = [s["link"] for s in sponsors]
check_results = check_sponsors(user_id, links)

unsubscribed = [
    item for item in check_results if item.get("status") == "unsubscribed"
]

```

```

if not unsubscribed:
    bot.answer_callback_query(call.id, "Отлично, подписка подтверждена!")
    handle_successful_subscription(
        user_id, chat_id, message_id=call.message.message_id
    )
else:
    bot.answer_callback_query(
        call.id, "Вы подписались не на все каналы!", show_alert=True
    )

```

```

# Кнопка "👤 Профиль"
@bot.message_handler(func=lambda message: message.text == "👤 Профиль")
def profile_handler(message):
    user_id = message.from_user.id
    balance, _, _, _ = get_user(user_id)
    ref_count = get_referral_count(user_id)

    text = (
        f"👤 Ваш ID {user_id}\n"
        f"💰 Ваш баланс "{balance}" Gram\n"
        f"👥 Рефералов {ref_count}"
    )

    markup = types.InlineKeyboardMarkup()
    btn_withdraw = types.InlineKeyboardButton(
        text="💸 Вывести", callback_data="start_withdraw"
    )
    markup.add(btn_withdraw)

    bot.send_message(message.chat.id, text, reply_markup=markup)

```

```
# Обработка нажатия на инлайн-кнопку "💎 Вывести"
@bot.callback_query_handler(func=lambda call: call.data == "start_withdraw")
def callback_withdraw_start(call):
    user_id = call.from_user.id
    balance, _, _, _ = get_user(user_id)

    user_states[user_id] = "waiting_for_withdraw_amount"

    text = f"🔄 Введите сколько хотите вывести\n💰 Ваш баланс {balance} Gram"
    bot.answer_callback_query(call.id)
    bot.send_message(call.message.chat.id, text)
```

```
# Кнопка "🎁 Бонус"
@bot.message_handler(func=lambda message: message.text == "🎁 Бонус")
def daily_bonus_handler(message):
    user_id = message.from_user.id
    user_states.pop(user_id, None)
    balance, last_bonus_str, _, _ = get_user(user_id)

    now = datetime.now()

    if last_bonus_str:
        last_bonus_time = datetime.fromisoformat(last_bonus_str)
        next_bonus_time = last_bonus_time + timedelta(hours=24)

        if now < next_bonus_time:
            remaining = next_bonus_time - now
            hours = int(remaining.total_seconds() // 3600)
            minutes = int((remaining.total_seconds() % 3600) // 60)

            bot.send_message(
                message.chat.id,
                f"❌ Вы уже забрали ежедневный бонус...\n⌚ Попробуйте через {hours}ч"
                f"{minutes}м",
            )
            return

    new_balance = balance + 500
    update_user_balance_and_bonus(user_id, 500, now.isoformat())

    bot.send_message(
        message.chat.id,
        f"🎉 Вы получили 500 Gram.\n"
        f"🎁 За сбор ежедневного бонуса\n"
        f"💰 Ваш текущий баланс: \"{new_balance}\".\n"
        f"⌚ Возвращайтесь завтра и получайте каждый день 500 Gram",
    )
```



```

# Кнопка "👤 Пригласить друга"
@bot.message_handler(func=lambda message: message.text == "👤 Пригласить друга")
def invite_friend_handler(message):
    user_id = message.from_user.id
    user_states.pop(user_id, None)
    bot_info = bot.get_me()
    ref_link = f"https://t.me/{bot_info.username}?start={user_id}"

    text = (
        f"{ref_link}\n\n"
        "Приглашайте друзей по этой ссылке и получайте 💰 огромное"
        " вознаграждение!\n\n"
        "В виде 7.000 GRAM!\n\n"
        "Награду вы получите после подписки на всех спонсоров"
    )
    bot.send_message(message.chat.id, text)

# --- АДМИН ПАНЕЛЬ ---
@bot.message_handler(commands=["admin"])
def admin_panel_cmd(message):
    if message.from_user.id != ADMIN_ID:
        return

    markup = types.InlineKeyboardMarkup(row_width=2)
    btn_ban = types.InlineKeyboardButton(text="🚫 Забанить", callback_data="adm_ban")
    btn_unban = types.InlineKeyboardButton(
        text="✅ Разбанить", callback_data="adm_unban"
    )
    btn_add = types.InlineKeyboardButton(
        text="➕ Добавить баланс", callback_data="adm_add_bal"
    )
    btn_sub = types.InlineKeyboardButton(
        text="➖ Снять баланс", callback_data="adm_sub_bal"
    )
    markup.add(btn_ban, btn_unban, btn_add, btn_sub)

    bot.send_message(
        message.chat.id,
        "👑 **Панель Администратора**\nВыберите действие:",
        reply_markup=markup,
        parse_mode="Markdown",
    )

@bot.callback_query_handler(

```

```

    func=lambda call: call.data.startswith("adm_")
    and call.from_user.id == ADMIN_ID
)
def admin_callbacks(call):
    action = call.data
    chat_id = call.message.chat.id

    if action == "adm_ban":
        user_states[ADMIN_ID] = "waiting_for_ban_id"
        bot.answer_callback_query(call.id)
        bot.send_message(chat_id, "Введите ID пользователя, которого нужно ЗАБАНИТЬ:")
    elif action == "adm_unban":
        user_states[ADMIN_ID] = "waiting_for_unban_id"
        bot.answer_callback_query(call.id)
        bot.send_message(
            chat_id, "Введите ID пользователя, которого нужно РАЗБАНИТЬ:"
        )
    elif action == "adm_add_bal":
        user_states[ADMIN_ID] = "waiting_for_add_id"
        bot.answer_callback_query(call.id)
        bot.send_message(
            chat_id, "Введите ID пользователя, которому нужно ДОБАВИТЬ баланс:"
        )
    elif action == "adm_sub_bal":
        user_states[ADMIN_ID] = "waiting_for_sub_id"
        bot.answer_callback_query(call.id)
        bot.send_message(
            chat_id, "Введите ID пользователя, у которого нужно СНЯТЬ баланс:"
        )
)

```

Обработка текстовых вводов для админки и вывода

@bot.message_handler(func=lambda message: message.from_user.id in user_states)

def process_states(message):

user_id = message.from_user.id

chat_id = message.chat.id

state = user_states[user_id]

text = message.text.strip()

Логика вывода средств

if state == "waiting_for_withdraw_amount":

user_states.pop(user_id, None)

try:

amount = int(text)

except ValueError:

bot.send_message(

chat_id, "❌ Пожалуйста, введите корректное число для вывода."

)

```

return

balance, _, _, _ = get_user(user_id)

if amount < 50000:
    bot.send_message(
        chat_id, "❌ Заявка на вывод отклонена\n📈 Минимум 50.000 Gram"
    )
    return

if amount > balance:
    bot.send_message(
        chat_id, "❌ Заявка на вывод отклонена\n💰 Недостаточно Средств"
    )
    return

cursor.execute(
    "UPDATE users SET balance = balance - ? WHERE user_id = ?",
    (amount, user_id),
)
conn.commit()

cursor.execute("SELECT balance FROM users WHERE user_id = ?", (user_id,))
new_balance = cursor.fetchone()[0]

bot.send_message(
    chat_id,
    f"✅ Заявка на вывод создана\n"
    f"👉 -{amount} Gram\n"
    f"💰 Ваш текущий баланс "{new_balance}" Gram\n"
    f"🕒 Ожидайте вывод средств",
)

try:
    user_info = bot.get_chat(user_id)
    who = f"@{user_info.username}" if user_info.username else str(user_id)
    admin_text = (
        f"🔔 Новая заявка на вывод!\n"
        f"Подал: {who} (ID: {user_id})\n"
        f"Заявка на: {amount} Gram"
    )
    bot.send_message(ADMIN_ID, admin_text)
except Exception as e:
    print(f"Не удалось отправить уведомление админу: {e}")
return

# Логика Админ-панели (только для ADMIN_ID)
if user_id == ADMIN_ID:

```

```

if state == "waiting_for_ban_id":
    try:
        target_id = int(text)
        set_ban_status(target_id, 1)
        user_states.pop(user_id, None)
        bot.send_message(
            chat_id,
            f"✅ Пользователь с ID `{target_id}` успешно забанен.",
            parse_mode="Markdown",
        )
    except ValueError:
        bot.send_message(chat_id, "❌ Ошибка. Введите числовой ID.")

elif state == "waiting_for_unban_id":
    try:
        target_id = int(text)
        set_ban_status(target_id, 0)
        user_states.pop(user_id, None)
        bot.send_message(
            chat_id,
            f"✅ Пользователь с ID `{target_id}` разбанен.",
            parse_mode="Markdown",
        )
    except ValueError:
        bot.send_message(chat_id, "❌ Ошибка. Введите числовой ID.")

elif state == "waiting_for_add_id":
    try:
        target_id = int(text)
        admin_temp_data[ADMIN_ID] = target_id
        user_states[ADMIN_ID] = "waiting_for_add_amount"
        bot.send_message(
            chat_id,
            f"Введите количество Gram, которое нужно ДОБАВИТЬ пользователю"
            f" `{target_id}`:",
            parse_mode="Markdown",
        )
    except ValueError:
        bot.send_message(chat_id, "❌ Ошибка. Введите числовой ID.")

elif state == "waiting_for_add_amount":
    try:
        amount = int(text)
        target_id = admin_temp_data.get(ADMIN_ID)
        new_bal = add_balance(target_id, amount)
        user_states.pop(user_id, None)
        bot.send_message(
            chat_id,

```

```

        f"✅ Успешно добавлено {amount} Gram пользователю `{target_id}`.\nНовый"
        f" баланс: {new_bal} Gram",
        parse_mode="Markdown",
    )
except ValueError:
    bot.send_message(chat_id, "❌ Ошибка. Введите число.")

elif state == "waiting_for_sub_id":
    try:
        target_id = int(text)
        admin_temp_data[ADMIN_ID] = target_id
        user_states[ADMIN_ID] = "waiting_for_sub_amount"
        bot.send_message(
            chat_id,
            f"Введите количество Gram, которое нужно СНЯТЬ у пользователя"
            f" `{target_id}`:",
            parse_mode="Markdown",
        )
    except ValueError:
        bot.send_message(chat_id, "❌ Ошибка. Введите числовой ID.")

elif state == "waiting_for_sub_amount":
    try:
        amount = int(text)
        target_id = admin_temp_data.get(ADMIN_ID)
        new_bal = add_balance(target_id, -amount) # отнимаем сумму
        user_states.pop(user_id, None)
        bot.send_message(
            chat_id,
            f"✅ Успешно снято {amount} Gram у пользователя `{target_id}`.\nНовый"
            f" баланс: {new_bal} Gram",
            parse_mode="Markdown",
        )
    except ValueError:
        bot.send_message(chat_id, "❌ Ошибка. Введите число.")

if __name__ == "__main__":
    print("Бот с админ-панелью запущен!")
    bot.infinity_polling()

```